Proyecto Final: Backend API REST con SQL Server para una Aplicación SaaS

Escuela de Tecnologías de la Información y Comunicación

Universidad Latina de Costa Rica

BTI23 – Bases de Datos II

Ciclo Lectivo VI

Nota: El proyecto representa el 60% de la nota final del curso. Se trabaja en grupos de máximo tres estudiantes. La entrega y defensa oral se realiza en la semana 15.

Contents

1 Descripción General **2**

2 Elección del Dominio **2**

3 Arquitectura del Sistema **3**

3.1 Capa de Base de Datos 3

3.1.1 Esquema relacional 3

3.1.2 Control de acceso 3

3.1.3 Stored procedures, funciones y triggers 4

3.1.4 Transacciones explícitas 4

3.1.5 Optimización 4

3.2 Capa API REST 5

4 Entregables y Etapas de Revisión **5**

5 Rúbrica de Evaluación **7**

6 Consideraciones Técnicas **8**

Descripción General

El proyecto consiste en el diseño e implementación de un backend completo para una aplicación de tipo Software as a Service (SaaS). La premisa central del curso determina la naturaleza del proyecto: toda la lógica de negocio debe residir en la base de datos, implementada mediante procedimientos almacenados (stored procedures), funciones y triggers en Microsoft SQL Server. El API REST actúa únicamente como capa de transporte entre cualquier cliente y la base de datos.

El grupo deberá producir cuatro entregables. Primero, un esquema relacional implementado en SQL Server con control de acceso configurado. Segundo, un API REST en el lenguaje o framework de preferencia del grupo, cuya única responsabilidad es recibir peticiones HTTP y ejecutar los stored procedures correspondientes. Tercero, un cliente de demostración, que puede ser una aplicación web, de escritorio, móvil o simplemente una colección de Postman, para demostrar la funcionalidad completa del sistema. Cuarto, un manual técnico y una presentación oral de defensa.

Ninguna lógica de negocio debe existir en el código del API. Validaciones, cálculos, transacciones y reglas de negocio deben estar en la base de datos. El servidor de aplicaciones únicamente puede autenticar la petición HTTP, invocar el stored procedure correspondiente y serializar la respuesta hacia el cliente.

Elección del Dominio

Cada grupo propone libremente una idea de negocio tipo SaaS como dominio del proyecto. No existe un tema predefinido; el grupo puede elegir cualquier sector siempre que la propuesta cumpla los siguientes criterios de viabilidad.

El modelo relacional debe tener al menos ocho entidades. El sistema debe contemplar roles de usuario claramente diferenciados, por ejemplo administrador, cliente y operador. Deben existir operaciones que requieran transacciones explícitas, como reservas, pagos o transferencias de inventario. La complejidad del dominio debe justificar el uso de optimización de consultas.

El grupo debe presentar su propuesta al docente durante la semana 1 para aprobación. Una vez

aprobado el dominio no puede cambiarse. El docente puede sugerir ajustes si considera que la complejidad no es la adecuada para los objetivos del curso.

Arquitectura del Sistema

El sistema sigue una arquitectura de tres capas. El cliente, que puede ser cualquier aplicación web, de escritorio o móvil, se comunica con el API REST mediante peticiones HTTP con carga útil en formato JSON. El API recibe la petición, verifica la autenticación y ejecuta el stored procedure correspondiente en SQL Server mediante `EXEC usp_...`. SQL Server procesa toda la lógica y devuelve el resultado, que el API serializa y retorna al cliente. El API no toma ninguna decisión de negocio; es un intermediario transparente.

Capa de Base de Datos

Esta es la capa principal del proyecto y la que concentra la mayor parte de la evaluación. Se divide en cinco componentes obligatorios.

Esquema relacional

El grupo debe diseñar e implementar el modelo relacional completo con un diagrama Entidad-Relación aprobado por el docente. El esquema debe estar normalizado hasta Tercera Forma Normal como mínimo e incluir restricciones de integridad referencial mediante llaves foráneas, restricciones `CHECK`, restricciones `UNIQUE` y columnas `NOT NULL` donde corresponda. Los tipos de datos de cada columna deben ser los apropiados para el dominio.

Control de acceso

El grupo debe crear al menos tres logins y usuarios en SQL Server que representen los roles de la aplicación, por ejemplo `app_admin`, `app_user` y `app_readonly`. A cada usuario se le asignan privilegios de objeto, como `EXECUTE` sobre stored procedures y `SELECT` o `INSERT` sobre vistas, siguiendo el principio de mínimo privilegio. El API debe conectarse a SQL Server con el usuario de menor privilegio que le permita operar. El uso del usuario `sa` o de cualquier cuenta con permisos de `dbo` en el entorno de producción está prohibido. El esquema de control de

acceso debe estar documentado en el manual técnico.

Stored procedures, funciones y triggers

Cada operación expuesta por el API, ya sea de creación, lectura, actualización, eliminación o generación de reportes, debe estar implementada como un stored procedure independiente. Los stored procedures deben utilizar parámetros de entrada y salida, manejo de errores con `TRY...CATCH` y retornar conjuntos de resultados o códigos de estado según corresponda.

Adicionalmente el proyecto debe incluir al menos dos funciones, escalares o de tabla, que sean utilizadas dentro de los stored procedures. También debe incluir al menos un trigger que audite una operación crítica o refuerce una regla de negocio que no pueda garantizarse solo con restricciones declarativas.

Transacciones explícitas

Al menos tres stored procedures deben implementar transacciones explícitas mediante `BEGIN TRANSACTION`, `COMMIT` y `ROLLBACK`. Las transacciones deben aplicarse en operaciones donde la consistencia sea crítica, como el registro de un pago que simultáneamente actualiza el inventario y genera una factura. Cada transacción debe contar con manejo de errores dentro del bloque `TRY...CATCH` que ejecute el rollback correspondiente ante cualquier falla.

Optimización

El grupo debe identificar al menos tres consultas críticas del sistema, es decir, las más frecuentes o las de mayor costo estimado, y documentar su plan de ejecución antes de intervenir. A partir de ese análisis deben proponer e implementar mejoras concretas, como la creación de índices no agrupados, índices compuestos o índices sobre columnas filtradas. El plan de ejecución después de la optimización debe documentarse junto con la comparación del costo estimado antes y después de cada intervención.

Capa API REST

El API puede implementarse con cualquier lenguaje o framework, por ejemplo Node.js con Express, Python con FastAPI, C# con .NET o Java con Spring Boot. Sus responsabilidades son las siguientes.

Autenticación con JSON Web Token (JWT): el endpoint de inicio de sesión invoca un stored procedure de validación de credenciales y, si es exitoso, genera y retorna un token JWT firmado con una expiración máxima de ocho horas. Autorización basada en roles: cada endpoint verifica el rol codificado en el JWT antes de ejecutar el stored procedure correspondiente. Enrutamiento: cada ruta HTTP se mapea directamente al stored procedure adecuado sin agregar lógica de negocio en el API. Serialización: el resultado del stored procedure se convierte a JSON para la respuesta HTTP. Manejo de errores: los errores retornados por SQL Server se traducen a los códigos HTTP apropiados, como 400, 401, 403, 404 o 500.

Entregables y Etapas de Revisión

El proyecto cuenta con cuatro revisiones parciales durante el cuatrimestre. El docente proporciona retroalimentación en cada revisión y el avance acumulado es requisito para la entrega final.

Revisión	Semana	Avance esperado
R1	3	Propuesta aprobada, modelo ER preliminar, base de datos creada en SQL Server, usuarios y roles configurados.
R2	5	Al menos el 40% de los stored procedures implementados, incluyendo los de autenticación. El API debe poder ejecutar al menos un stored procedure.
R3	8	El 80% de los stored procedures completos, transacciones explícitas implementadas.
R4	12	Sistema completo funcionando. Análisis de planes de ejecución documentado antes de optimizar.
Final	15	Todos los entregables completos, manual técnico y demostración en vivo.

Para la entrega final en la semana 15 el grupo debe presentar los siguientes materiales.

Un repositorio de código en GitHub o plataforma equivalente, con acceso otorgado al docente, que contenga el script SQL completo para crear y poblar la base de datos, el código fuente del API REST y un archivo `README.md` con las instrucciones de instalación y ejecución.

Un manual técnico en formato PDF de mínimo 20 páginas que incluya la descripción del dominio y la justificación de la idea SaaS, el diagrama ER final, el diccionario de datos, el esquema de control de acceso con usuarios y privilegios, la descripción de cada stored procedure, función y trigger, la documentación de las transacciones explícitas, el análisis de optimización con planes de ejecución comparados y la guía de uso del API con ejemplos de peticiones y respuestas en JSON.

Una presentación oral y defensa de máximo 30 minutos con demostración en vivo del sistema. El docente formulará preguntas a cada integrante de forma individual sobre cualquier componente del proyecto, por lo que todos los miembros deben dominar la totalidad del trabajo.

Rúbrica de Evaluación

El proyecto tiene una ponderación del 60% de la nota final del curso. El desglose interno de la calificación del proyecto es el siguiente.

Componente	**Porcentaje**
Control de acceso (usuarios, roles y privilegios en SQL Server)	15%
Stored procedures, funciones y triggers	25%
Transacciones explícitas	15%
Análisis y propuestas de optimización	15%
API REST (integración, JWT y manejo de errores)	15%
Manual técnico	10%
Presentación oral y defensa	5%
Total	**100%**

Los descriptores cualitativos de calificación son los siguientes. Una calificación de excelente, entre 91 y 100, corresponde a un proyecto que implementa correctamente el control de acceso, cubre toda la lógica del dominio mediante stored procedures, utiliza transacciones explícitas y robustas, y presenta propuestas de optimización con evidencia de planes de ejecución comparados. Una calificación de notable, entre 81 y 90, corresponde a un proyecto con deficiencias menores en uno de los cuatro componentes técnicos principales. Una calificación de bueno, entre 71 y 80, corresponde a deficiencias en dos de los cuatro componentes. Una calificación de insuficiente, igual o menor a 70, corresponde a deficiencias en tres o más componentes o a un sistema que no puede demostrarse en vivo.

Las siguientes condiciones resultan en reprobación automática del proyecto: que el script SQL no se ejecute limpiamente en SQL Server, que la lógica de negocio esté implementada en el código del API en lugar de en la base de datos, que el grupo no pueda demostrar el sistema funcionando durante la presentación oral, que algún integrante no pueda responder preguntas sobre cualquier parte del proyecto durante la defensa, o que se detecte plagio o uso de código de otros grupos sin

atribución.

Consideraciones Técnicas

En cuanto a SQL Server, el grupo debe utilizar SQL Server 2019 o una versión superior, o SQL Server Express para entornos locales. Todos los objetos de la aplicación deben crearse dentro de un esquema propio del proyecto, sin usar el esquema `dbo` directamente. Se recomienda nombrar los stored procedures con un prefijo consistente, por ejemplo `usp_Clientes_Crear` o `usp_Pedidos_Procesar`. Cada stored procedure debe iniciar con `SET NOCOUNT ON` y debe incluir un comentario de cabecera con el nombre, descripción, parámetros, autor y fecha.

En cuanto al API REST, la cadena de conexión a SQL Server no debe estar en el código fuente; debe almacenarse en variables de entorno o en un archivo `.env` excluido del repositorio. Se recomienda documentar todos los endpoints con Swagger u OpenAPI.

En cuanto al trabajo en equipo, el uso de control de versiones con Git es obligatorio. El historial de commits debe evidenciar la contribución de cada integrante del grupo. El docente puede solicitar a cualquier integrante que explique o modifique cualquier parte del proyecto en cualquier momento de la defensa.